Trabajo Práctico Integrador

Lógica y Teoría de Grafos

Introducción

En las materias vimos grafos como una herramienta para modelar problemas: caminos mínimos, árboles recubridores, flujos, coloreo, recorridos eulerianos. En la práctica los grafos suelen estar dados: los nodos, las aristas y sus pesos son parte de los problemas. En la realidad esto nunca es así. ¿Cuánto tarda un camión en recorrer una calle a las 18 h con lluvia? ¿Qué tan probable es que un estudiante apruebe una materia? ¿Cuántos estudiantes van a querer rendir dos exámenes el mismo día? Esos números hay que estimarlos a partir de datos!

Este trabajo práctico integra tres cosas que trabajamos durante el año:

1. Machine Learning: entrenar un modelo sencillo que a partir de datos históricos prediga una cantidad o una categoría relevante para el problema.
2. Teoría de grafos: usar esas predicciones como entrada (aristas, capacidades, pesos, etc.) de un algoritmo de grafos.
3. Desarrollo web: mostrar todo eso como una aplicación web, con un backend que haga los cálculos y un frontend donde se pueda ver y probar el resultado.

La idea es poder completar un proyecto:

problema $\rightarrow$ datos $\rightarrow$ modelo predictivo $\rightarrow$ grafo con pesos $\rightarrow$ algoritmo de grafos $\rightarrow$ aplicación web

Si bien queremos ver que cada parte funcione por separado, también que están conectadas. Si el modelo predice algo, el algoritmo de grafos se adapta y eso se puede ver en la aplicación.

Consignas generales (aplican a todos los TPs)

Modalidad

- El trabajo se realiza en grupos de 2 personas.
- Cada grupo elige uno de los cinco TPs propuestos en este documento.
- Alternativamente, el grupo puede proponer un TP propio. La propuesta debe cumplir todas las consignas generales de esta sección (problema, modelo predictivo, algoritmo de grafos que use las predicciones, backend y frontend) y tiene que ser aprobada por los profes antes de empezar.
- **Fecha de entrega: 08/11/2026.**

Qué tiene que hacer cada grupo

Independientemente del TP elegido, el trabajo consta de las siguientes etapas:

1. Entender el problema. Elegir un problema y entender qué significa: quién lo tiene, qué decisión se quiere tomar, qué información haría falta para tomarla bien y por qué un grafo es una buena forma de modelarlo.
2. Elegir variables y entrenar un modelo. Identificar qué features sirven para predecir algo útil para el problema, y entrenar un modelo. Puede ser un problema de clasificación o de regresión. Los modelos que pueden usar son:
   - Regresión lineal.
   - Regresión logística.
   - KNN.
   - SVM.
   - Árboles de decisión.

   Se pueden entrenar y comparar varios. Se deben seguir las prácticas que vimos en clase.
3. Implementar algoritmos de grafos que usen las predicciones. Los algoritmos pedidos en cada TP tienen que recibir como entrada lo que predice el modelo (por ejemplo, como peso de las aristas). Si el modelo cambia su predicción, el resultado del algoritmo tiene que poder cambiar.
4. Armar un backend y un frontend. El backend contiene toda la implementación (preparación de datos, modelo, algoritmos de grafos) y la expone mediante una API. El frontend consume esa API y permite que los profes veamos cómo funciona, cambiar parámetros y ver el resultado.
5. **Documentar el uso de IA.**

Datasets reales o sintéticos

Para entrenar el modelo hace falta un dataset. Pueden encararlo de dos formas, ambas válidas:

- Dataset real adaptado: Buscar un dataset público (datos abiertos, Kaggle, UCI, datos de gobierno, etc.) relacionado con el problema y adaptarlo: renombrar o combinar columnas, filtrar, completar lo que falte. Hay que citar la fuente y explicar qué transformaciones se hicieron.
- Dataset sintético: Generar los datos artificialmente.

Seguramente se preguntarán qué es un dataset sintético. Es un conjunto de datos que no fue sacado del mundo real, sino fabricado para que se parezca a datos reales. Se usa cuando los datos reales no existen, son privados o son difíciles de conseguir. Pueden usar herramientas de IA para generar el dataset sintético (por ejemplo, pidiéndole que escriba un script generador). En ese caso:

- Conviene que la IA genere un script que produzca los datos, y no que escriba las filas a mano. El script es reproducible, se puede ajustar y permite generar la cantidad de datos que haga falta.
- El script (o el procedimiento usado) se entrega junto con el código.
- Tienen que declarar las reglas que usaron para generar los datos (qué variable influye en cuál y cómo). Un dataset sin estructura (por ejemplo, valores completamente al azar) hace que el modelo no pueda aprender nada útil.
- Tengan en cuenta que el modelo va a descubrir las reglas que ustedes pusieron. Es esperable y está bien, pero hay que mencionarlo.

Las tablas de campos que aparecen en cada TP son puramente orientativas. Indican el tipo de

información esperada, pero pueden agregar, quitar o renombrar columnas si lo necesitan.

Herramientas sugeridas

Pueden elegir los lenguajes, librerías y frameworks. Lo único que pedimos es que el backend exponga sus resultados mediante una API y que el frontend corra en el navegador y la consuma. Como punto de partida pueden usar estas herramientas:

- Python para el backend con paquetes extra:
  - Machine Learning: `scikit-learn`, junto con `pandas` para manejar los datos.
  - Grafos: `networkx` para representar el grafo (nodos, aristas, atributos) y para leerlo o escribirlo.
  - API: Flask o FastAPI.
- TypeScript con React.

Uso de inteligencia artificial

El uso de herramientas de IA no solo está permitido sino que es altamente recomendado y esperado. Son herramientas que van a usar en su vida profesional, y saber usarlas bien es parte de lo que queremos que aprendan.

Por "usarlas bien" entendemos que la IA es una herramienta que ustedes dirigen y no un reemplazo de entender el trabajo. Algunos usos razonables: generar el dataset sintético, armar el esqueleto del frontend, explicar un error, proponer una forma de visualizar el grafo, revisar si una implementación es correcta. Lo que no puede pasar es que no sepan explicar qué hace su propio código. En la defensa oral les vamos a preguntar sobre cualquier parte del trabajo así que estén atentos.

El informe debe incluir una sección de uso de IA donde expliquen:

- Qué herramientas usaron.
- Para qué partes del trabajo las usaron, y para cuáles no.
- Algunos ejemplos concretos de prompts que les resultaron útiles.
- Casos en los que la IA se equivocó o propuso algo incorrecto, y cómo se dieron cuenta.
- Una reflexión breve: ¿en qué los ayudó y en qué no?

Entregables

1. Código fuente en un repositorio Git (backend y frontend), con historial de commits que refleje el trabajo incremental del grupo (no se aceptan repositorios con un único commit final).
2. Informe que incluya:
   - Descripción del problema y decisiones de diseño.
   - Descripción del dataset utilizado (fuente o reglas de generación sintética, features elegidas y por qué).
   - Metodología de entrenamiento del modelo: partición de datos, métricas usadas, resultados obtenidos, análisis de errores.
   - Descripción del algoritmo de grafos implementado.
   - Explicación de cómo se integran ambas partes (qué salida del modelo se usa como entrada del algoritmo de grafos, y por qué esa transformación es razonable).

- Capturas de pantalla del frontend funcionando.
- Sección de uso de IA (ver Uso de inteligencia artificial).
- Limitaciones conocidas y posibles extensiones.

3. Demo funcionando end-to-end (backend + frontend).
4. Instrucciones de instalación y ejecución (`README.md`) que permitan levantar el proyecto desde otro entorno.

Ruteo de última milla con predicción de demanda

Contexto

Una empresa de logística quiere optimizar sus entregas diarias. Actualmente calcula rutas usando distancias fijas entre puntos, pero el tiempo de viaje varía mucho según la hora del día, el día de la semana, el clima y otros factores. La empresa los contrata para construir un sistema que prediga el tiempo de viaje real en cada tramo de la ciudad y use esa predicción para calcular rutas de entrega óptimas.

Objetivo

Construir un sistema que combine un modelo de regresión (para estimar tiempos de viaje) con algoritmos de grafos (para calcular rutas óptimas sobre esos tiempos estimados).

Datos

Se puede modelar la ciudad como un grafo dirigido:

- Nodo: intersección o punto de entrega con coordenadas (lat, lon).
- Arista: tramo de calle entre dos nodos con dirección (si corresponde a una calle de sentido único).

El grupo debe generar (o conseguir) un dataset histórico de viajes con, al menos, estas columnas:

Campo	Tipo	Descripción
`tramo_id`	id	identificador de la arista
`hora_dia`	entero (0–23)	hora de inicio del viaje
`dia_semana`	categórica	lunes a domingo
`clima`	categórica	soleado / lluvia / tormenta
`es_feriado`	booleano	—
`velocidad_promedio_historica`	numérico	velocidad promedio histórica del tramo
`tiempo_real_viaje`	numérico	**variable a predecir**

Requisitos

1. **Modelo predictivo:** entrenar un modelo de regresión que dado un tramo y un contexto (hora, día, clima), prediga el tiempo de viaje esperado. Analizar métricas (MAE, RMSE, R^2) sobre un conjunto de test separado.
2. **Actualización dinámica de pesos:** el grafo debe poder recalcular el peso de cada arista usando la predicción del modelo para un momento dado (elegido por el usuario en el frontend).
3. **Grafos:** implementar Dijkstra para calcular la ruta más rápida entre un origen y un destino usando los pesos predichos.

Frontend

- Mapa interactivo que muestre el grafo de calles.
- Selector de origen/destino y de momento (hora, día, clima) para calcular y visualizar la ruta más rápida.

- Visualización de la ronda de entregas calculada, con el orden de visita marcado sobre el mapa.
- Comparación visual entre la ruta calculada con pesos estáticos (distancia) y con pesos predichos (tiempo dinámico), para que se note el valor agregado del modelo.

Diseño de infraestructura de red

Contexto

Una cooperativa de servicios de agua necesita planificar el tendido de su red en una zona nueva. Quiere decidir qué conexiones construir para llegar a todos los barrios con el menor costo posible, y verificar si la capacidad de esa red alcanza para la demanda que va a haber en unos años. La cooperativa los contrata para construir un sistema que proyecte la demanda futura de cada barrio y use esa proyección para diseñar y validar la red.

Objetivo

Construir un sistema que combine un modelo de regresión (para proyectar la demanda futura de cada zona) con algoritmos de grafos (para diseñar la red de menor costo y verificar si su capacidad alcanza para esa demanda).

Datos

Se puede modelar la zona como un grafo no dirigido:

- Nodo: barrio, manzana o edificio con coordenadas.
- Arista: posible tendido entre dos nodos cercanos, con un costo de construcción (por ejemplo, en función de la distancia) y una capacidad máxima de transporte.

El grupo debe generar (o conseguir) un dataset de zonas con, al menos, estas columnas:

Campo	Tipo	Descripción
`nodo_id`	id	—
`latitud`, `longitud`	numérico	—
`poblacion`	numérico	habitantes o unidades servidas
`tipo_zona`	categórica	residencial / comercial / industrial
`consumo_historico_mensual`	numérico	consumo promedio actual
`crecimiento_poblacional_estimado`	numérico	% anual proyectado
`consumo_futuro`	numérico	**variable a predecir**

Requisitos

1. **Modelo predictivo:** entrenar un modelo de regresión que, a partir de las características de cada zona, proyecte su consumo futuro (por ejemplo, a 5 años). Analizar métricas (MAE, RMSE, R^2) sobre un conjunto de test separado.
2. **Demanda en el grafo:** usar la demanda proyectada por el modelo como la demanda de cada nodo.
3. **Grafos:** implementar Kruskal o Prim para calcular la red de menor costo que conecte todos los nodos, y Ford-Fulkerson (o Edmonds-Karp) sobre esa red para verificar si su capacidad cubre la demanda proyectada, identificando las aristas saturadas.

Frontend

- Visualización de los tendidos posibles, con la red de menor costo resaltada.

- Vista de la red con el flujo sobre cada arista, resaltando las aristas saturadas o cerca de su capacidad máxima.
- Selector de escenario de crecimiento (por ejemplo, +10 %, +30 %) para ver cómo cambia el diagnóstico de capacidad.

Planificador curricular

Contexto

La dirección de una carrera quiere ofrecer a sus estudiantes una herramienta que sugiera qué materias cursar. Hoy la única guía es el plan de estudios sugerido, que es igual para todos. La diferencia es que el riesgo de desaprobar una materia depende mucho del historial de cada estudiante: su promedio, cuántas materias cursa a la vez, cómo le fue en las correlativas. La carrera los contrata para construir un sistema que prediga ese riesgo y use esa predicción para sugerir el orden de cursada más seguro, respetando las correlatividades.

Objetivo

Construir un sistema que combine un modelo de clasificación (para estimar la probabilidad de aprobar cada materia) con algoritmos de grafos (para calcular la secuencia de cursada de menor riesgo sobre el grafo de correlatividades).

Datos

Se puede modelar el plan de estudios como un grafo dirigido acíclico (DAG):

- Nodo: materia.
- Arista: correlatividad (de la materia que hay que tener aprobada a la que la requiere).

Pueden usar el plan de estudios de una carrera real o armar uno realista, con correlatividades interesantes (algunas materias con más de una correlativa).

Para el modelo, el grupo debe generar (o conseguir) un dataset histórico de cursadas con, al menos, estas columnas:

Campo	Tipo	Descripción
`estudiante_id`, `materia_id`	id	—
`promedio_previo`	numérico	promedio del estudiante hasta ese momento
`cant_materias_aprobadas_previas`	numérico	—
`cant_materias_cursando_simultaneo`	numérico	—
`nota_promedio_correlativas`	numérico	notas en las correlativas de la materia
`es_recursada`	booleano	—
`resultado`	categórica	aprobó / desaprobó / abandonó: **variable a predecir**

Requisitos

1. **Modelo predictivo:** entrenar un modelo de clasificación que, dado el perfil de un estudiante y una materia, estime la probabilidad de aprobarla. Analizar métricas (accuracy, precisión, recall, matriz de confusión) sobre un conjunto de test separado.
2. **De probabilidades a pesos:** definir y justificar cómo se transforma la probabilidad predicha en un peso del grafo. Al simular que el estudiante aprueba o desaprueba una materia, los pesos se recalculan con el perfil actualizado.
3. **Grafos:** implementar Dijkstra para calcular, desde las materias que el estudiante ya aprobó,

el camino de materias de menor riesgo acumulado respetando las correlatividades.

Frontend

- Visualización del grafo de correlatividades, con las materias coloreadas según el riesgo estimado para el estudiante.
- Secuencia sugerida resaltada sobre el grafo.
- Formulario para cargar un historial (o elegir un estudiante de ejemplo) y ver cómo cambia la sugerencia.
- Opción de simular que el estudiante aprueba o desaprueba una materia y ver la secuencia recalculada.

Planificación de mesas de examen

Contexto

La secretaría académica de una facultad tiene que armar el cronograma de mesas de examen. El problema es evitar que dos materias con muchos estudiantes en común caigan el mismo día y turno, porque esos estudiantes tendrían que elegir cuál rendir. La secretaría los contrata para construir un sistema que estime qué pares de materias van a tener conflicto y use esa estimación para armar un cronograma con la menor cantidad de turnos posible.

Objetivo

Construir un sistema que combine un modelo predictivo (para estimar el conflicto entre cada par de materias) con algoritmos de grafos (para asignar turnos de examen sin superposiciones mediante coloreo).

Datos

Se puede modelar el problema como un grafo de conflictos no dirigido:

- Nodo: materia que se rinde en el período.
- Arista: conflicto entre dos materias (estudiantes que querrían rendir ambas), con un peso igual la complejidad del conflicto.

El grupo debe generar (o conseguir) un dataset histórico por par de materias con, al menos, estas columnas:

Campo	Tipo	Descripción
`materia_id_1`, `materia_id_2`	id	—
`cant_estudiantes_compartidos_historico`	numérico	—
`mismo_anio`	booleano	si las materias son del mismo año de la carrera
`misma_carrera`	booleano	—
`cant_estudiantes_afectados`	numérico	**variable a predecir** (regresión)
`hubo_conflicto`	booleano	**variable a predecir** (alternativa, clasificación)

Requisitos

1. **Modelo predictivo:** entrenar un modelo de regresión (¿cuántos estudiantes se ven afectados?) o de clasificación (¿hay conflicto o no?) sobre los pares de materias. Analizar métricas adecuadas al tipo de modelo sobre un conjunto de test separado.
2. **Construcción del grafo:** usar la predicción del modelo para decidir qué aristas forman el grafo de conflictos (por ejemplo, solo los pares con conflicto predicho por encima de un umbral) y con qué peso.
3. **Grafos:** implementar un algoritmo de coloreo greedy que asigne un turno (color) a cada materia, de forma que dos materias en conflicto no compartan turno.

Frontend

- Visualización del grafo de conflictos, con las materias coloreadas según el turno asignado.

- Vista de cronograma (tipo calendario o grilla) con las materias que se rinden en cada turno.
- Selector del umbral de conflicto, para ver cómo cambian el grafo y la cantidad de turnos necesarios.

Recolección de residuos

Contexto

El área de higiene urbana de un municipio quiere optimizar el recorrido de sus camiones recolectores. A diferencia de un problema de entregas, donde importa visitar puntos, acá el camión tiene que recorrer calles completas, y no todas las calles necesitan recolección con la misma urgencia. El municipio los contrata para construir un sistema que prediga qué tramos tienen más prioridad y use esa predicción para calcular un recorrido que los cubra a todos de forma eficiente.

Objetivo

Construir un sistema que combine un modelo predictivo (para estimar la prioridad de recolección de cada tramo) con algoritmos de grafos para calcular un recorrido que pase por todos los tramos prioritarios.

Datos

Se puede modelar la zona como un grafo no dirigido:

- Nodo: intersección.
- Arista: tramo de calle con su distancia o tiempo de recorrido.

El grupo debe generar (o conseguir) un dataset de tramos con, al menos, estas columnas:

Campo	Tipo	Descripción
`tramo_id`	id	identificador de la arista
`tipo_zona`	categórica	residencial / comercial / industrial
`cantidad_contenedores`	numérico	—
`dias_desde_ultima_recoleccion`	numérico	—
`historico_quejas_vecinales`	numérico	—
`prioridad_recoleccion`	categórica	alta / media / baja: **variable a predecir**

Requisitos

1. **Modelo predictivo:** entrenar un modelo de clasificación que, a partir de las características de un tramo, prediga su prioridad de recolección. Analizar métricas (accuracy, precisión, recall, matriz de confusión) sobre un conjunto de test separado.
2. **Selección de tramos:** definir un criterio (por ejemplo, prioridad media o alta) para decidir, según la predicción del modelo, qué tramos tiene que recorrer el camión.
3. **Grafos:** implementar el algoritmo de Hierholzer para construir un circuito que recorra todos los tramos seleccionados. Si el subgrafo no es euleriano, resolver el problema del cartero chino: encontrar los vértices de grado impar, calcular caminos mínimos entre ellos con Dijkstra, emparejarlos con costo mínimo, duplicar esos caminos y construir el circuito sobre el grafo resultante. Para el emparejamiento se puede usar una librería, aclarándolo en el informe.

Frontend

- Visualización del grafo de calles, con los tramos coloreados según la prioridad predicha.
- Visualización paso a paso del recorrido calculado, marcando los tramos que se recorren más de una vez.
- Costo total del recorrido.